Accuracy converges, firing rate does not

nb024 · 2 June 2026

The trained networks this entry uses are produced once in the shared training hub, nb022 (Training), and reused here rather than retrained.

Abstract

Trains PING and COBA from scratch for 100 epochs (three seeds each) and asks whether the firing rate converges once the accuracy has. It does not — not for COBA. Test accuracy plateaus by ≈ 15 epochs in both architectures, but PING's E rate locks to a tight ≈ 9 Hz attractor (cross-seed to 0.1 Hz) while COBA's keeps climbing through epoch 100. Accuracy is a point; for COBA, the rate is a manifold the optimiser drifts along at constant accuracy.

Method

Reads the shared θ_u = off baselines (coba and ping, three seeds each) from the training hub — nb022 (Training), which fixes the recipe (100 epochs, mem-mean readout, no rate regulariser) — and plots their per-epoch training history. The question, following nb041 / nb044: once test accuracy has plateaued, does the firing rate also settle? **Converged** means a last-10-epoch slope below 0.1 pp/ep (accuracy) or 0.05 Hz/ep (rate).

Results

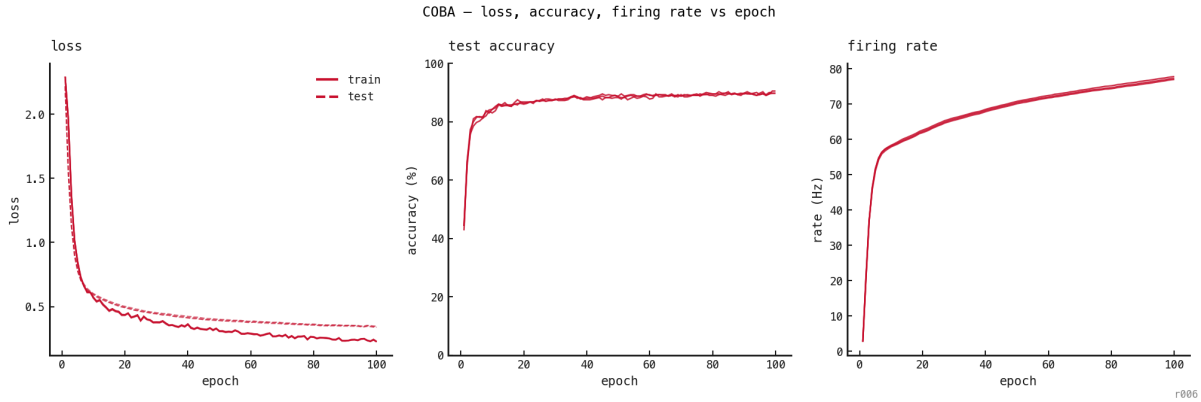

Figure 62: COBA, three seeds. Loss (train solid, test dashed) and **accuracy plateau by ≈ 15 epochs**, but the **E rate keeps climbing to ≈ 77 Hz and is still rising at epoch 100** — accuracy has converged, the rate has not.

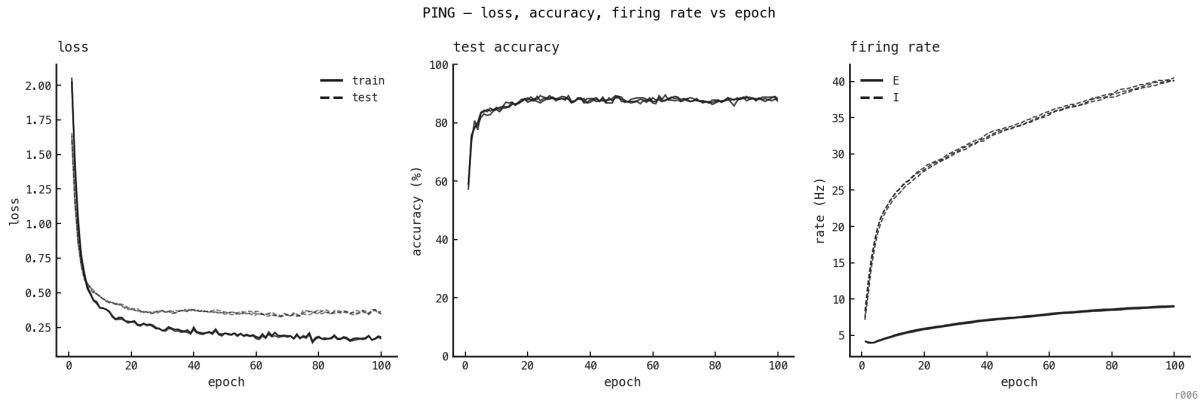

Figure 63: PING, three seeds. Loss and accuracy plateau by ≈ 15 epochs as in COBA, but here the **E rate locks flat at ≈ 4–9 Hz** — converged. (The I rate, dashed, keeps climbing to ≈ 40 Hz, driven by the still-growing input weights, the same force that drives COBA's E rate up.) The loop pins the excitatory rate; without it, COBA's drifts.

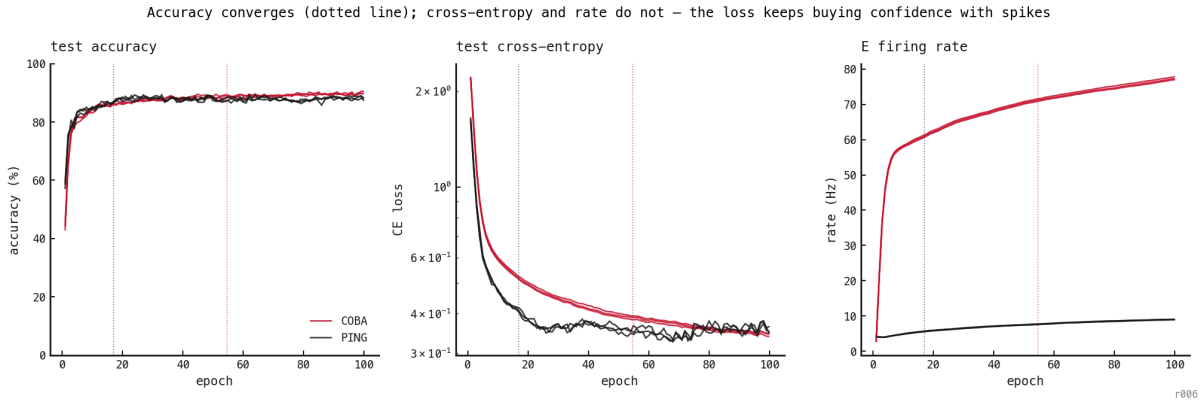

Figure 64: Test accuracy (left), test cross-entropy on a log axis (middle), and E firing rate (right) vs epoch; three seeds each, COBA red / PING black. Dotted verticals mark each model's accuracy-convergence epoch (first epoch within 1% of final accuracy). Accuracy is flat past ≈ epoch 20, yet **cross-entropy keeps falling well to its right** — COBA's is still dropping at epoch 100. COBA's E rate climbs ≈ 55 → 78 Hz in lockstep; PING reaches the same low loss early and **stays pinned near ≈ 9 Hz**.

Discussion

The rate climb is what the loss spends to keep gaining confidence. Cross-entropy stops at *certain*, not *correct*:

$$\mathrm{CE} = -\log p_y = \log\left(1 + \sum_{k \neq y} e^{z_k - z_y}\right)$$

- z_y, z_k — logits for the true class and class k
- p_y — softmax probability of the true class
- $m = z_y - \max_{k \neq y} z_k$ — decision margin

Accuracy needs only the *sign* of m; cross-entropy keeps shrinking with the *size* of the gaps. So past the convergence line the argmax is fixed but the loss still falls by widening margins — which means scaling logits up. With a mem-mean readout, $z \approx W_{\text{out}} \cdot (\text{E activity})$, so wider margins cost either weight or spikes. COBA takes the spike route (rate ≈ 55 → 78 Hz, still climbing);

PING sharpens its readout through the loop and holds ≈ 9 Hz. The loop buys confidence for free; without it, confidence costs spikes.

So the rate doesn't fail to converge — it tracks a loss that never stops rewarding margin. train.py now logs *test_margin*, *test_confidence* and *test_logit_scale* per epoch; once retrained, the prediction is that COBA's margin rises with its rate while PING's plateaus.

Next steps

Figure 3 infers confidence from the rate. The next run (cells retrained with the new logging) lets us measure it directly:

- Plot per-epoch **margin** $\langle m \rangle$, **confidence** $\langle p_y \rangle$ and **logit scale** vs epoch, COBA vs PING — the direct version of Figure 3′s middle panel.
- Overlay each against the E rate to test the lockstep: confidence up with rate for COBA, both flat for PING.
- Confirm the split quantitatively — does COBA's margin keep rising at epoch 100 (slope $>$ 0) while PING's plateaus (slope $\approx$ 0), matching the cross-entropy and rate slopes here.